

Sample Chapter: Core JDO ISBN 0-13-140731-7

The chapters of the book titled “Core JDO” are being made available for the purposes of review to allow the community to participate in the review and editing of the chapters of the book.

Copyright (c) 2002 Sun Microsystems, Inc. All rights reserved.

FOR PRIVATE USE ONLY: This material is the property of Sun Microsystems, Inc. and is not to be sold, reproduced in any form by any means, or generally distributed without the prior written authorization of Sun Microsystems Inc and Prentice Hall.

To add your comments

- Simply send your uninhibited comments and feedback to the authors at jdoobook@yahoogroups.com
- Or click and add a note using Acrobat where needed. E-mail the PDF back to the authors at jdoobook@yahoogroups.com
- If you would like to receive the chapters in an alternate format, have any questions, concerns or please contact the authors at jdoobook@yahoogroups.com

“I call architecture frozen music” -Johann Wolfgang von Goethe

Architecture Scenarios

*T*his chapter outlines the different types of application architectures that can be used with JDO, beyond the standalone, single threaded, Java applications that has been the focus of the book up to this point. Using JDO with these different architectures is not that much different from using JDO to build a standalone application. The concepts and APIs are all the same – the only difference is how `PersistenceManagerFactory`, `PersistenceManager` and `Transaction` instances are used.

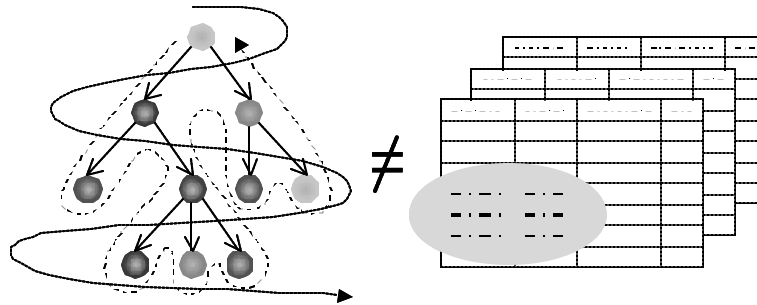
This chapter covers:

- When to use JDO and when to use JDBC
- The different types of datastores that can be used with JDO
- Using JDO with multi-tier applications
- Differences when using JDO in managed and unmanaged environments
- Multi-threaded programming with JDO

8.1 JDO VERSUS JDBC

Put in its simplest form JDO can be used to build any Java application that needs to access data stored a datastore, whatever the architecture of the Java application. To this extent, JDO and JDBC can be viewed as competing ways of accessing persistent data from Java. However, the difference between JDO and JDBC lies in JDO’s object perspective.

For data-centric applications, whose main purpose is to store and retrieve record-oriented data then JDBC is likely the best tool for the job. Record-oriented data can easily be



mapped directly to a relational model and an application can leverage the power of SQL for ad-hoc queries and data access. The focus is on the data.

For object-centric applications, whose main purpose is to process a complex graph or hierarchy of interrelated data then JDO is likely the best tool for the job. An object model can be used to represent the complex graph or hierarchy of interrelated data that the application can navigate through to complete its processing. The focus is on the object model.

Simply put, JDBC is for applications that need to think of data as relations (tables, rows and columns), JDO is for applications that need to think of data as interrelated objects. Figure 8-1 shows the difference in structure between object-centric and data-centric applications:

Figure 8-1 An Object-oriented versus Relational-oriented Application

8.2 RDBMS, ODBMS AND FLATFILES

Unlike JDBC, which is specific to databases that support SQL, JDO is datastore neutral. The same JDO application can be used with a relational database, an object database, a flat file or even an in-memory database. All that is required is a JDO implementation for the underlying datastore being used.

Although the application itself remains the same across databases, how a database is setup and used with a JDO implementation varies. The JDO specification does not define how a persistence capable class is actually mapped to its representation in the underlying

datastore, this is left as to the JDO implementation. Typically before a JDO application can be run the following steps need to be performed:

- Specify the mapping between the persistence capable classes and the datastore schema. JDO implementations typically use the “vendor-extension” element in the JDO metadata to specify the mapping between the persistence capable class and its representation in the datastore and would provide tools that can be used to generate this mapping.
- Create a database and define the schema. Again a JDO implementation would typically provide tools that can be used to define the schema in the database.

How each step is performed will vary between JDO implementations, an implementation’s documentation should be consulted for further details.

8.2.1 Using JDO with a Relational Database

There are three approaches to using JDO with a relational database:

- Define the persistence capable classes and generate the database schema
- Define the database schema and generate the persistence capable classes
- Define the persistence capable classes and database schema independently and define a mapping between the two.

The first approach is suitable for applications where there is no existing database and the application is free to define its own schema. The database schema can be generated from the persistence capable classes. This has the advantage that the application can leverage the power of object-orientation when defining its persistence capable classes. A JDO implementation’s tools would be used to automatically define the schema in the database for the persistence capable classes and create the mapping specification between the persistence capable classes and the schema.

The second approach is suitable for applications where a database already exists and the application has to use the existing schema. The persistence capable classes can be generated from the schema. This has the advantage that a JDO application can easily access existing data in a database, however the generated classes aren’t going to be particularly object-oriented in nature. A JDO implementation’s tools would be used to generate the Java classes from the database schema for the application and create the mapping specification between the persistence capable classes and the schema.

The third approach is suitable for applications where both the persistence capable classes and the database schema already exist. A mapping is specified between the persistence capable classes and the existing database schema. This has the advantage that a JDO application can leverage the power of object-orientation when defining its persistence capa-

ble classes but still access existing data in a database. A JDO implementation's tools would be used to define the mapping between the persistence capable classes and the database schema. This approach is the most complex of the three since defining a mapping between two arbitrary models is a difficult task. Typically for this approach to work the persistence capable classes and database schema have to be of a similar structure.

A JDO implementation that supports relational databases may not support all of the approaches outlined. A developer needs to choose an implementation that supports the approach required by their application.

8.2.2 Using JDO with an Object Database

Object databases differ from relational databases in that they do not use the relational model (tables, columns and rows) to store data, instead they use the object model (classes, fields and instances). Direct support for the object model means a JDO persistence capable class can be mapped directly to its object representation in the database.

The approach to using an object database with JDO is similar to the first approach outlined for using a relational database. The persistence capable classes are defined and the database schema is generated. A JDO implementation's tools would be used to define the schema in the database for the persistence capable classes. There is typically no mapping to specify since the persistence capable classes map one for one to the classes defined in the database.

8.2.3 Object versus Relational databases

As with most things there is no single technology that is best for all needs. Although there has been much heated debate over the years about the pros and cons of using object versus relational databases much of it has failed to recognize that the different technologies satisfy different needs. This doesn't necessarily make one technology better than the other it just means that a developer needs to understand which is most appropriate for the job in hand.

The primary strength of an object database is its ability to manage arbitrarily complex object models, models that include:

- Inheritance,
- Multi-valued fields (arrays of values),
- One-to-one, one-to-many and bi-directional relationships,
- Relationships that include semantics: sets (uniqueness); lists (ordering); maps (associative lookup). Where the relationships may be complex objects in themselves, perhaps containing hashed values for efficient in-memory lookup and retrieval.

Complex object models are typically hard to map to an efficient representation in a relational database and because an object database is able to directly represent a persistence capable class without any mapping a JDO application gets the following benefits:

- **Development** – additional metadata is not required to specify how a persistence capable class should be represented in the database. This may appear only a minor benefit, but as the number of persistence capable classes increases along with the complexity of those classes (use of inheritance and relationships) the additional metadata can add up.
- **Performance** – because the persistence capable classes are directly represented in the database there is less overhead to retrieve a persistent object (if using a relational database it is likely that the representation of a persistence capable class would require several tables). As the complexity of the classes increases typically the performance delta between relational and object databases increases.

Whilst a discussion of the relative merits of object and relational databases is beyond the scope of this book, a simple perspective is that an object database can be very effective as an application specific database whereas a relational database is much better choice as an enterprise-wide database. Essentially if an application persists data for its own needs only an object database can be a good candidate. If an application needs to persist data to be accessed enterprise-wide then a relational database is a good choice.

8.3 J2EE, RMI AND CORBA

This first part of the book has shown how JDO can be used to build a standalone Java application. However JDO can be used to build J2EE, RMI or CORBA applications also. In particular JDO has been specifically designed to work in conjunction with J2EE. Essentially wherever a Java application can directly use JDBC today, JDO could likewise be used.

In many ways JDO offers a programming model and approach to object persistence similar to Container Manager Persistence (CMP) from the Enterprise JavaBeans (EJB) specification. However unlike CMP, JDO keeps persistence orthogonal to the component architecture being used. This means that the power of object persistence can be taken advantage of directly within a Java application or a Servlet without incurring the overhead of the J2EE component architecture. Or used by any other non-J2EE distributed platforms like RMI and CORBA.

That is not to say that JDO has no role to play within EJB. In fact JDO combined with SessionBeans makes for a powerful architecture. SessionBeans provide the remote, component-level interface, leveraging the container services like resource pooling, connection and transaction management, whilst JDO provides portable lightweight object persistence.

Although the EJB 2.0 specification provides an enhanced specification for CMP that aims to address some of the performance issues of the previous version, from a development standpoint CMP is still an order of magnitude more complex than JDO to use.

With CMP a developer needs to:

- Provide a local and/or remote Home interface, which defines the methods used to locate instances of the EntityBean class.
- Provide a local and/or remote EntityBean interface, which defines the public methods of the EntityBean class.
- Provide a local and/or remote abstract class that implements the methods as declared in the associated EntityBean's interface. The class doesn't actually declare the fields of the EntityBean, instead it has to declare JavaBeans style getters and setters for each field. Fields and relationships to other EntityBeans's have to be declared in the deployment descriptor for the EntityBean.
- Provide a deployment descriptor that declares: the EntityBean's local and/or remote Home and EntityBean interfaces; the abstract class that implements the EntityBean's methods and the fields and relationships of the EntityBean.
- Deploy the EntityBean. At which point a concrete implementation of the EntityBean's abstract class is generated by the container along with the necessary code to store and retrieve the fields of the EntityBean to/from the underlying datastore.

When all the above has been done the EntityBean can be run and tested, however any issue that requires a code change requires a redeployment of the EntityBean. Also, the EntityBean itself can only be run within a J2EE container, although a local interface will at least allow an application within the same JVM to bypass the overhead of remote invocation.

With JDO a developer needs to:

- Provide a normal Java class. Fields are declared as normal, relationships are implemented using Java references and standard collection classes. Inheritance can be used.
- Provide an XML file containing the metadata for the persistence capable class. Since most of the metadata is derived, all that is really needed is the class name.
- Compile the class using a standard Java compiler and use a JDO compliant byte-code enhancement tool.

At this point the class can be used by any Java application, a Servlet or EJB Session Bean, RMI or CORBA application.

8.4 MANAGED AND UNMANAGED ENVIRONMENTS

The term managed and unmanaged has been used at various points in the book to differentiate between traditional, two-tier Java applications and multi-tier Java applications.

A two-tier Java application explicitly connects to the underlying datastore (which constitutes the second tier) and begins and ends its own local transactions. A two-tier application typically starts, does something at the bequest of a user and ends.

A multi-tier Java application requests connections from a connection manager and may delegate transaction control to an external coordinator. This coordinator begins and ends transactions implicitly, potentially coordinating multiple different data sources in a single distributed transaction. A multi-tier application typically runs continuously processing requests that it receives from other applications. Servlets, EJBs and applications that use RMI or CORBA are all examples of multi-tier applications.

8.4.1 Connection Management

To interact with the underlying datastore a JDO application needs to get a connection. A two-tier application typically requires a connection for the duration of the application, during which time it would begin and end many transactions. A multi-tier application typically requires a connection for the duration of a single transaction only. For this reason, multi-tier applications rely on resource pooling to avoid connections being continuously opened and closed. Instead connections, once opened are managed in a resource pool. Each time a multi-tier application requires a connection to process a request one is allocated from the pool, when finished the connection is given back to the pool. Although typically used by multi-tier applications, some two-tier applications can also benefit from resource pooling.

8.4.1.1 Two-tier Applications

A two-tier Java application constructs a `PersistenceManagerFactory` from which it gets one or more `PersistenceManager` instances which when finished with it closes. The act of getting a `PersistenceManager` instance and closing it may or may not correlate with physically opening and closing connections to the underlying datastore.

Depending on the JDO implementation being used and how it is configured the `PersistenceManagerFactory` may pool the underlying datastore connections or `PersistenceManager` instances. This means that when the application closes a `PersistenceManager` instance it doesn't physically close anything but rather just gives back the resources to the `PersistenceManagerFactory` to be re-used. All this is transparent to the application, which simply gets a `PersistenceManager` instance when it needs one and closes it when it is finished.

A JDO implementation may also allow use of external connection factories. If configured to use an external connection factory the `PersistenceManagerFactory` will rely on the external factory to pool datastore connections. It requests a connection when needed and

returns it to the factory when finished. See the `ConnectionFactory` and `ConnectionFactory-Name` properties on `PersistenceManagerFactory` for more details on using external connection factories.

Connection pooling is more typically useful for multi-tier applications, but some multi-threaded, two-tier applications may benefit from pooling also. Particularly those that can have many threads where a thread doesn't necessarily need a `PersistenceManager` instance all of the time. By pooling, an application can reduce the resources it needs at any given point in time.

8.4.1.2 Multi-tier Applications

A multi-tier application looks up a `PersistenceManagerFactory` (typically using JNDI) from which it gets one or more `PersistenceManager` instances.

If used in a J2EE runtime environment a `PersistenceManagerFactory` instance is no longer responsible for connection management and instead relies on the J2EE container to pool connections and manage the quality of service associated with use of those connections.

Most JDO implementations use the J2EE Connector Architecture (JCA) as the means by which the `PersistenceManagerFactory` instance interacts with the J2EE container. The JCA resource adapter provided with a JDO implementation is configured and deployed into a J2EE runtime environment. The `PersistenceManagerFactory` instance retrieved via JNDI then delegates connection management to the J2EE container.

The JDO specification does not mandate that a JDO implementation use JCA, an implementation may choose a different way of integrating with a J2EE container. Although how the JDO implementation is configured and deployed may differ, the application itself remains the same (it looks up a `PersistenceManagerFactory` instance from which it gets `PersistenceManager` instances).

Non-J2EE multi-tier applications need to ensure that the JDO implementation being used implements its own connection pooling.

8.4.2 Transaction Management

Transactions provide the context within which persistent objects are stored in the datastore and retrieved again. A two-tier application typically begins and ends transactions explicitly whilst a multi-tier application may delegate transaction control to an external coordinator, in which case the coordinator begins and ends transactions implicitly.

8.4.2.1 Two-tier Applications

A two-tier application gets a `Transaction` instance from a `PersistenceManager`, and uses it to explicitly begin and end transactions.

8.4.2.2 Multi-tier Applications

A multi-tier application can delegate transaction control to an external coordinator, typically the Java Transaction Service (JTS). When using an external coordinator transactions are automatically controlled.

If used in a J2EE runtime environment a `PersistenceManagerFactory` instance ensures that a `PersistenceManager` instance is associated with the externally coordinated transaction (if there is one) when it is retrieved. This is relevant primarily for EJBs. If a method has been declared as transactional, then when the bean method gets its `PersistenceManager` instance from the `PersistenceManagerFactory`, the `PersistenceManagerFactory` returns a `PersistenceManager` instance associated with the external transaction. The external coordinator then implicitly begins and ends the transaction.

A multi-tier application doesn't have to delegate transaction management to an external coordinator. In this case the application explicitly begins and ends transactions via the `Transaction` interface. This is relevant for Servlets or EJBs with non-transactional methods.

As per connection management most JDO implementations use JCA as the means by which the `PersistenceManagerFactory` interacts with the J2EE container. However this is not mandated and an implementation may use a different mechanism.

Non-J2EE applications that require external coordination of transactions need to ensure that the JDO implementation being used supports the Java Transaction APIs (JTA) so it can be integrated with JTS.

8.5 MULTI-THREADED APPLICATIONS

Many of the examples in the book so far have been of the simplest type of JDO application. The application runs, gets a `PersistenceManager` instance from a `PersistenceManagerFactory`, begins a transaction, does something and ends the transaction. This is a single threaded, single `PersistenceManager` application.

JDO supports much more sophisticated applications than this. An application can essentially get as many `PersistenceManager` instances at a time as available resources allow. Each `PersistenceManager` instance represents a connection to the datastore, a transaction context and a cache of persistent objects. From the application perspective each `PersistenceManager` instance is independent of the other and from the datastore perspective each `PersistenceManager` instance is essentially a separate user. An application can be multi-threaded and concurrently access the datastore using different `PersistenceManager` instances.

8.5.1 Multi-threaded Programming

When using multiple threads, a developer needs to give thought to how threads interact with a `PersistenceManager` instance. A `PersistenceManager` instance is essentially a shared resource, therefore care needs to be taken when multiple threads concurrently access it.

In fact not only is the `PersistenceManager` instance itself a shared resource but since the `PersistenceManager` acts as a factory for `Query`, `Extent` and `Transaction` instances any of those instances are likewise shared resources. Also any persistent objects associated with the `PersistenceManager` instance are shared resources.

If multiple threads are going to concurrently access a `PersistenceManager` instance or any associated instances (`Query`, `Extent` and so on) then the “Multithreaded” property of the `PersistenceManager` instance needs to be true. This ensures that the JDO implementation synchronizes internally and ensures that concurrent access doesn’t cause internal data structures to be corrupted. This synchronization may however add additional overhead.

Beyond this the application needs to ensure that multiple threads interact with the `PersistenceManager` instance correctly. Whilst multiple threads can access persistent objects at the same time, it is still the applications responsibility to synchronize this access. This is no different from normal multi-threaded programming where multiple threads are concurrently accessing the same Java objects. Also care needs to be taken when using various methods on `PersistenceManager` that affect the lifecycle state of persistent objects. For example, an application needs to ensure that persistent objects being evicted or refreshed by one thread are not being used by another.

When completing a transaction the lifecycle state of all persistent objects associated with the `PersistenceManager` instance change (they all transition to being “hollow”). An application needs to ensure that only one thread calls `commit` or `rollback` at a time and that at this time other threads are not accessing any persistent objects associated with the `PersistenceManager` instance.

8.5.2 Simplified Multi-threaded Programming

Although an application may be multi-threaded, if there is only a single thread accessing a `PersistenceManager` instance or its associated instances at a given point in time then as far as the JDO runtime is concerned this is equivalent to the application being single threaded. Because each `PersistenceManager` instance is independent of any other, multiple threads can concurrently access different `PersistenceManager` instances without conflict and the “Multithreaded” `PersistenceManager` property does not need to be true.

8.6 SUMMARY

This chapter has provided an overview of how JDO can be used with different application architectures beyond a standalone Java application. The next chapters will focus specifically on J2EE and how JDO can be used to build J2EE applications.